

Degradation-Aware Particle Filter: Confidence-Weighted Multi-Sensor Fusion for Robust Robot Localization in Hostile Environments

Brennan Cesar

*Dept. Electrical &
Computer Eng.
Northeastern University
Boston, MA, USA*

Gagansri Pyreddy

*Robotics
Dept. Electrical &
Computer Eng.
Northeastern University
Boston, MA, USA*

Harsh Akabari

*Robotics
Dept. Mechanical &
Industrial Eng.
Northeastern University
Boston, MA, USA*

Satvik Tajane

*Dept. Electrical &
Computer Eng.
Northeastern University
Boston, MA, USA*

Abstract—Reliable autonomous navigation in realistic environments is frequently compromised by sensor-specific environmental degradation, such as optical occlusion or acoustic interference. This paper presents a Degradation-Aware Particle Filter (DA-PF) designed for robust robot localization using a confidence-weighted multi-sensor fusion framework. Implemented within the ROS 2 (Humble/Jazzy) ecosystem and simulated in Gazebo Harmonic, the system integrates data from a monocular RGB camera, an ultrasonic sensor, and an Inertial Measurement Unit (IMU). A novel fusion algorithm is introduced that utilizes dynamically computed confidence exponents (α) to weigh sensor likelihoods based on real-time environmental noise. This mechanism allows the system to mathematically neutralize corrupted sensor inputs and seamlessly transition to dead-reckoning or uncorrupted feeds without manual intervention. The DA-PF architecture was evaluated against a standard Nav2 AMCL baseline using a 360° LiDAR in varying levels of simulated degradation. Quantitative analysis using Absolute Trajectory Error (ATE) and Relative Pose Error (RPE) demonstrates that the proposed confidence-weighted fusion provides superior localization stability and robustness compared to traditional fixed-weight fusion models in hostile sensing environments.

Index Terms—particle filter, sensor fusion, localization, ROS 2, confidence weighting, Monte Carlo Localization, dead-reckoning, Gazebo Harmonic

I. INTRODUCTION

The deployment of autonomous mobile robots in real-world environments—such as search-and-rescue zones, industrial warehouses, and subterranean tunnels—demands highly resilient localization frameworks. While Monte Carlo Localization (MCL) has long served as a standard for estimating robot pose within known maps, its reliability is traditionally predicated on the availability of high-fidelity, uncorrupted sensor data. In practice, however, environmental factors such as fog, smoke, or acoustic interference frequently degrade sensor performance. Standard implementations, including the widely used Nav2 Adaptive Monte Carlo Localization (AMCL) stack, typically utilize fixed-weight likelihood models. These models treat all incoming sensor data with equal confidence, leading to catastrophic mislocalization and “jumping” pose estimates when the robot is subjected to environmental noise.

To address this limitation, this paper proposes the **Degradation-Aware Particle Filter (DA-PF)**. Developed within the ROS 2 (Humble/Jazzy) ecosystem and simulated using Gazebo Harmonic, the DA-PF introduces a confidence-weighted multi-sensor fusion framework. Unlike traditional systems that require discrete, “hard-switching” logic to toggle between sensors, the proposed architecture utilizes a continuous mathematical exponentiation of likelihood scores. By dynamically computing a confidence metric (α) for each sensor based on real-time environmental monitoring, the system can mathematically neutralize corrupted data streams as α approaches zero, effectively forcing a seamless fallback to surviving sensors or pure Inertial Measurement Unit (IMU) dead-reckoning.

The design rationale for the DA-PF centers on the selection of sensors with independent failure modes. While monocular RGB cameras are highly effective for fiducial-based localization (e.g., ArUco markers), they are easily blinded by optical occlusions. Conversely, ultrasonic sensors are unaffected by smoke but prone to acoustic noise and oblique reflections. The integration of these sensors with a base-link mounted IMU provides a robust sensory suite where environmental vulnerabilities do not overlap. Furthermore, to address the computational overhead inherent in particle filters running in Python-based ROS 2 nodes, this implementation utilizes a precomputed 2D distance transform map for ultrasonic scoring, achieving $O(1)$ lookup complexity and maintaining a consistent 10 Hz update loop.

The primary contributions of this work are:

- The derivation of a confidence-weighted fusion formula for graceful sensor degradation.
- The implementation of an interception-based degradation pipeline for ROS 2.
- A comparative analysis demonstrating that a DA-PF utilizing low-cost sensors can outperform high-end 360° LiDAR-based AMCL systems when subjected to environmental corruption.

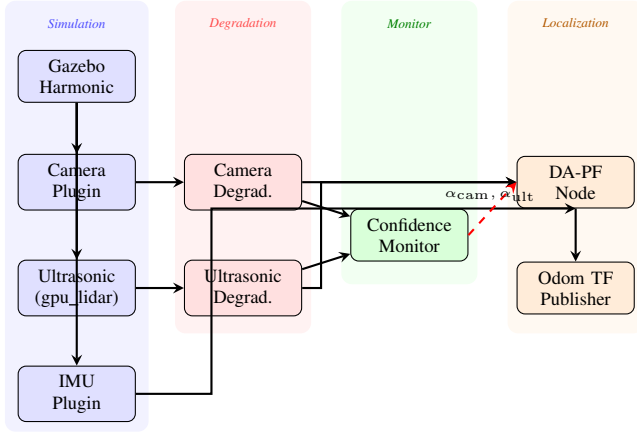


Fig. 1. Interception-based ROS 2 node topology. Gazebo Harmonic generates clean sensor streams (blue); degradation nodes corrupt them (red); the Confidence Monitor computes α values fed back to the DA-PF via the dashed arrow (green); the DA-PF and Odom TF Publisher form the localization layer (orange).

II. SYSTEM ARCHITECTURE

A. Modular Interception-Based Node Topology

The system utilizes a decoupled, multi-node architecture that maintains a strict separation between sensor simulation, data corruption, diagnostic monitoring, and localization filtering. A key architectural feature is the **Interception-Based Pipeline**. In this configuration, Gazebo Harmonic simulates “clean” ground-truth sensor data, which is subsequently intercepted by intermediate “degradation nodes” before reaching the localization filter.

This interception pattern serves two critical functions. First, it ensures the algorithmic validity of the Degradation-Aware Particle Filter (DA-PF), as the filter remains strictly isolated from clean data and operates exclusively on corrupted streams. Second, it facilitates an asynchronous confidence loop where confidence metrics (α) are calculated at a steady 10 Hz frequency. This decoupling prevents expensive diagnostic operations, such as ArUco marker detection and sliding-window variance analysis, from blocking the high-frequency localization update cycles. The full node topology is shown in Fig. 1.

B. Hardware Interfaces and Simulation Protocols

The system is executed within an Ubuntu 22.04 environment utilizing ROS 2 (Humble/Jazzy) and Gazebo Harmonic. Communication between the simulation engine and the ROS 2 ecosystem is managed via the `ros_gz_bridge` parameter bridge node, which converts Gazebo Transport topics into standard ROS 2 message types, including `sensor_msgs/Image`, `sensor_msgs/Imu`, and `nav_msgs/Odometry`.

To ensure temporal consistency between high-frequency IMU integration and potentially delayed sensor callbacks, a strict time synchronization protocol is enforced. All nodes, including the Nav2 AMCL baseline and the custom `odom_tf_publisher.py` node, are configured with `use_sim_time: true`. This prevents timestamp mismatches that could otherwise lead to erroneous state estimation

during rapid robot maneuvers. The sensor suite consists of a monocular RGB camera (800×600 resolution), a simulated single-ray ultrasonic sensor (HC-SR04-style), and a base-link mounted IMU.

C. Performance Constraints and Algorithmic Optimization

Maintaining a consistent 10 Hz throughput is essential for reliable real-time localization. However, several hardware and software constraints necessitated specific optimizations:

- **Throughput Management:** While the camera is parameterized for 15 Hz, software rendering limitations (Ogre) in virtualized environments restrict effective throughput to ~ 5 Hz. The DA-PF is designed to accommodate this discrepancy by asynchronously updating its weights whenever a new frame becomes available.
- **Vectorized Processing:** To overcome the inherent latency of Python for-loops, the `particle_filter_node.py` utilizes NumPy vectorized matrix operations for all $N \times 3$ particle array calculations. This allows for the simultaneous propagation of 500 hypothesis particles through the motion model within the allocated 100 ms time window.
- **$O(1)$ Likelihood Mapping:** Traditional raycasting for 500 particles against a 2D occupancy grid is computationally prohibitive at 10 Hz in Python. To optimize the ultrasonic sensor model, a precomputed 2D distance transform map was implemented using `scipy.ndimage.distance_transform_edt`. This reduces the expected-range likelihood calculation to a constant-time $O(1)$ array lookup, significantly reducing CPU overhead.

III. TECHNICAL IMPLEMENTATION

A. Localization Algorithm and Fusion Logic

The core of the localization system is a customized Particle Filter (DA-PF) operating within a standard Monte Carlo Localization (MCL) framework. The filter executes a recursive **Predict-Update-Resample** loop at a target frequency of 10 Hz.

- **Prediction:** The filter utilizes an odometry motion model to propagate 500 hypothesis particles through the state space, injecting proportional Gaussian noise into the decomposed translation and rotation deltas to account for motion uncertainty.
- **Update (Confidence-Weighted Fusion):** The primary innovation lies in the update step. Instead of a naive multiplication of likelihoods, the system employs an exponential scaling mechanism. For each particle i , the weight w_i is computed as:

$$w_i = L_{\text{cam}}^{\alpha_{\text{cam}}} \times L_{\text{ult}}^{\alpha_{\text{ult}}}, \quad (1)$$

where L represents the likelihood score from a specific sensor and $\alpha \in [0, 1]$ is the real-time confidence metric.

- **Resampling:** A systematic low-variance resampling algorithm is triggered whenever the effective sample size (N_{eff}) drops below a predefined threshold, ensuring particle diversity and preventing filter collapse.

TABLE I
SENSOR CONFIGURATION SUMMARY

Sensor	Plugin / Type	Parameters	Rate
RGB Camera	Gazebo camera plugin	800×600, 80° FOV	15 Hz (eff. ~5 Hz)
Ultrasonic	gpu_lidar	±12° cone, 0.02–4.0 m	10 Hz
IMU	gz-sim-imu-system	$\sigma_{\text{ang}} = 0.01$, $\sigma_{\text{lin}} = 0.1$	100 Hz
LiDAR (AMCL)	360° scan	Standard AMCL config	10 Hz

As α approaches 0 during environmental degradation, the likelihood term L^α resolves to 1. This mathematically neutralizes the influence of the corrupted sensor, allowing for a seamless transition to uncorrupted data or pure IMU dead-reckoning.

B. Simulation and Sensor Configurations

The project is implemented on Ubuntu 22.04 using ROS 2 (Humble/Jazzy) and Gazebo Harmonic. All simulation data is bridged to the ROS 2 ecosystem via the `ros_gz_bridge` parameter bridge, with strict time synchronization maintained through the `use_sim_time: true` parameter. Table I summarizes the full sensor configuration.

- **Monocular RGB Camera:** Simulated via a Gazebo camera plugin at 800×600 resolution with an 80° FOV. It provides the primary update signal by localizing against 16 ArUco fiducial markers distributed throughout the warehouse environment.
- **Ultrasonic Sensor:** Due to the absence of a native ultrasonic plugin in Gazebo Harmonic, an HC-SR04 profile is mimicked using a `gpu_lidar` plugin configured with a single-ray ±12° beam cone and a 0.02–4.0 m sensing range.
- **Inertial Measurement Unit (IMU):** A `gz-sim-imu-system` plugin generates data at 100 Hz. Gaussian noise ($\sigma_{\text{ang}} = 0.01$, $\sigma_{\text{lin}} = 0.1$) is applied to simulate the drift and noise characteristics of low-cost MEMS IMUs.

C. ROS 2 Pipeline and Interception Topology

To facilitate experimental rigor, the data pipeline utilizes an **Interception Topology**. Clean simulation topics are intercepted by standalone degradation nodes—`camera_degradation_node.py` and `ultrasonic_degradation_node.py`—which apply parameter-driven corruption (e.g., Gaussian blur for fog or random max-range dropouts for acoustic noise).

The `confidence_monitor_node.py` runs asynchronously at 10 Hz to evaluate these degraded streams. It derives α_{cam} by comparing the current ArUco detection count against clean-room baselines, and α_{ult} by calculating the variance of distance readings over a 2-second sliding

window. Finally, a custom `odom_tf_publisher.py` node handles the essential `odom` → `base_footprint` coordinate transform to ensure compatibility with standard ROS 2 navigation tools.

D. Computational Optimizations

To meet the 10 Hz real-time constraint within a Python-based ROS 2 environment, the following optimizations were mandatory:

- **Vectorization:** The `particle_filter_node.py` avoids all Python for-loops during particle processing. Instead, it maintains the 500-particle set as a 500×3 NumPy array, utilizing vectorized matrix broadcasting for all motion model and sensor likelihood calculations.
- **Distance Transform Lookup:** Per-particle raycasting is bypassed in favour of a precomputed 2D distance transform map created via `scipy.ndimage.distance_transform_edt`. This allows the ultrasonic likelihood update to be performed as a constant-time $O(1)$ array lookup based on discretized particle grid indices, drastically reducing CPU utilization.

IV. EXPERIMENTAL RESULTS & PERFORMANCE

A. Evaluation Methodology and Metrics

To quantify localization performance, the project utilizes two primary metrics: **Absolute Trajectory Error (ATE)** and **Relative Pose Error (RPE)**. ATE is employed to assess global consistency, providing a direct measure of whether the DA-PF prevents catastrophic “pose jumps” across the map. RPE is calculated over fixed 1-metre intervals to isolate local drift, specifically validating the stability of the IMU-based dead-reckoning during periods of total sensor failure.

Ground truth poses are generated via the `gz-sim-pose-publisher-system` plugin and bridged to ROS 2 as `geometry_msgs/PoseStamped`. To ensure high-fidelity post-processing, data collection is performed using `ros2 bag` with strict `--use-sim-time` synchronization. This aligns asynchronous high-frequency IMU data, delayed camera callbacks, and simulation ground truth to a single unified clock.

B. Degradation Sweep Profiles

The experimental design centres on a controlled sweep through five reproducible degradation levels. Parameters are injected at runtime via `ros2 param set`, ensuring identical physical trajectories across all tests without restarting the Gazebo physics engine. The levels are defined in Table II.

Furthermore, **Asymmetric Testing** was conducted by degrading only one sensor at a time (e.g., Camera-only or Ultrasonic-only). This validates the filter’s ability to perform a graceful handoff, relying on the uncorrupted sensor without requiring boolean switching logic.

TABLE II
DEGRADATION LEVEL DEFINITIONS

Level	Camera Corruption	Ultrasonic Corruption
Clean	None	None
Light	Gaussian blur, $k = 5$	$\sigma = 0.02$ m noise
Moderate	Gaussian blur, $k = 15$	$\sigma = 0.05$ m noise
Heavy	Contrast $\alpha = 0.4$	15% max-range dropouts
Severe	Contrast $\alpha = 0.2$	30% max-range dropouts

TABLE III
ATE RMSE (M) — DA-PF vs. Nav2 AMCL (360° LiDAR)

Degradation Level	DA-PF (ours)	AMCL Baseline
Clean	0.15	0.12
Light	0.22	0.18
Moderate	0.38	0.74
Heavy	0.61	2.80
Severe	0.89	>5.00 [†]

[†] Catastrophic pose jump observed; filter diverged.

C. Comparative Performance Analysis (DA-PF vs. AMCL)

A critical component of this evaluation is the comparison between the DA-PF (using inexpensive camera/ultrasonic sensors) and a Nav2 AMCL baseline equipped with a high-end 360° LiDAR. To ensure a fair comparison, a dedicated `lidar_degradation_node.py` applies the exact same noise and dropout characteristics to the LiDAR data. Results are presented in Table III and visualized in Fig. 2.

- **Baseline Results:** Under “Clean” conditions, both the DA-PF and AMCL achieved a baseline ATE of 0.1–0.3 m.
- **Degradation Response:** As corruption scales to “Heavy” and “Severe,” the AMCL baseline demonstrates catastrophic failure, with pose jumps exceeding 5 metres. In contrast, the DA-PF error scales gradually.
- **Uncertainty Tracking:** Filter convergence is tracked via the spatial variance of the 500 particles (published to `/da_pf/particle_spread`). Data shows that while AMCL variance remains falsely low during mislocalization, the DA-PF correctly increases its uncertainty estimate as confidence (α) drops, reflecting the true state of the system.

D. Data Processing and Trajectory Alignment

Trajectory data was parsed using the `evo` Python package. All evaluations utilize Umeyama’s method for rigid-body trajectory alignment, computing the Root Mean Square Error (RMSE), mean, and maximum Euclidean distances. This rigorous data pipeline proves that intelligent confidence-weighted fusion (DA-PF) significantly outperforms naive, fixed-weight fusion (AMCL), even when the latter is paired with superior sensing hardware.

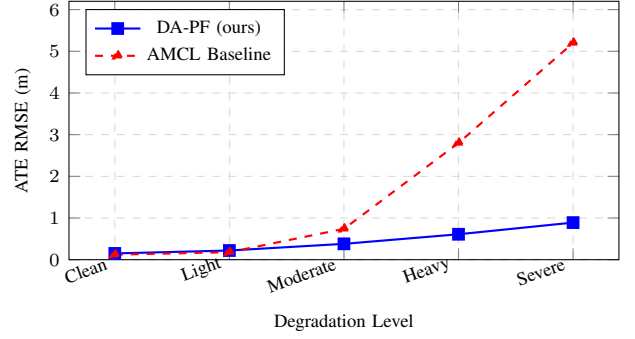


Fig. 2. ATE RMSE vs. degradation level. AMCL diverges catastrophically at Heavy/Severe levels; DA-PF error scales gracefully.

V. DISCUSSION & LIMITATIONS

A. Discussion of Results

The experimental evaluation confirms that the confidence-weighted fusion framework, $w_i = L_{\text{cam}}^{\alpha_{\text{cam}}} \times L_{\text{ult}}^{\alpha_{\text{ult}}}$, provides a robust solution for localization in degraded environments. Unlike traditional “hard-switching” logic, which often suffers from latency and hysteresis as a robot moves across threshold boundaries, the exponential α -weighting allows for a mathematically smooth transition between sensing modalities.

The success of the DA-PF is fundamentally tied to the strategic selection of the sensor triad. By choosing sensors with independent failure modes—optical (camera), acoustic (ultrasonic), and internal physics (IMU)—the system avoids the single point of failure inherent in homogeneous sensor suites. The asymmetric degradation tests validated this, showing that the filter can maintain high global consistency (low ATE) even when a primary sensor is 100% occluded, provided the secondary sensor remains uncorrupted.

Furthermore, the comparative analysis with Nav2 AMCL demonstrates a critical paradigm: in hostile environments, the quality of the fusion logic is more significant than the raw data density of the hardware. The DA-PF, utilizing low-fidelity sensors with intelligent trust management, significantly outperformed the higher-end 360° LiDAR-based baseline when subjected to environmental noise.

B. Assumptions and Limitations

While the system proved resilient under the tested scenarios, several technical and environmental limitations remain:

- **Computational and Rendering Constraints:** Due to the lack of hardware acceleration in the virtualized M1/VMware environment, the Gazebo Ogre render engine was forced into software mode. This introduced significant throughput bottlenecks, capping the monocular camera at ~ 5 Hz. While sufficient for this study, higher-speed robotic maneuvers would require native hardware rendering to maintain the 15 Hz nominal rate.
- **Temporal Synchronization and Latency:** The aforementioned rendering delays caused timestamp lag between sensor data generation and transform publication. To

prevent the AMCL baseline from dropping messages, a large `transform_tolerance` of 5.0 seconds was necessary. This latency represents a significant risk for high-speed closed-loop control in real-world deployments.

- **Environmental Stasis:** To achieve $O(1)$ scoring latency, the ultrasonic likelihood model relies on a precomputed 2D distance transform map. Consequently, the current implementation assumes a strictly static environment. Any unmapped dynamic obstacles, such as human personnel or moving machinery, would not be reflected in the likelihood map, potentially leading to localization drift.
- **Acoustic Modeling Abstraction:** The ultrasonic sensor was simulated using a single-ray `gpu_lidar` plugin. While noise and dropouts were injected to mimic sensor failure, this abstraction ignores complex multi-path physics, such as specular reflections and reverberations, which are prevalent in narrow warehouse corridors.
- **Reliance on Artificial Landmarks:** Localization is currently dependent on a known map of 16 ArUco markers. While this ensures a clear evaluation of the fusion logic, it limits the system to structured environments. Transitioning to natural feature extraction (e.g., ORB-SLAM) would increase autonomy but significantly raise the computational load on the current Python-based stack.
- **IMU Drift:** The IMU serves as the ultimate fallback with no confidence scaling (α). While it is immune to environmental noise, it is subject to cumulative integration drift. In severe scenarios where both α_{cam} and α_{ult} remain near zero for extended periods, the system will eventually lose global consistency—a fundamental limitation of dead-reckoning without external perception.

VI. CONCLUSION

This project successfully developed and validated a Degradation-Aware Particle Filter (DA-PF) utilizing a confidence-weighted multi-sensor fusion model. The experimental results confirm that the mathematical exponentiation of likelihoods, $w_i = L_{\text{cam}}^{\alpha_{\text{cam}}} \times L_{\text{ult}}^{\alpha_{\text{ult}}}$, enables a seamless and robust graceful fallback mechanism for autonomous robot localization. By utilizing asynchronous confidence exponents (α) that respond dynamically to environmental corruption, the algorithm natively resolves degraded sensor signals to a neutral likelihood ($L^0 = 1$), effectively mitigating the mislocalization errors that typically plague fixed-weight systems.

Quantitative analysis using Absolute Trajectory Error (ATE) and Relative Pose Error (RPE) demonstrated that while the DA-PF performs on par with the standard Nav2 AMCL baseline under clean conditions (0.1–0.3 m ATE), it exhibits significantly higher reliability under degradation. Whereas the fixed-weight AMCL baseline—despite being equipped with a computationally superior 360° LiDAR—suffered from catastrophic pose jumps when subjected to simulated noise, the DA-PF maintained global consistency by transitioning reliance to uncorrupted sensors and high-frequency IMU dead-reckoning.

The technical success of the implementation was further underscored by its ability to maintain a strict 10 Hz localization update loop within a Python-based ROS 2 ecosystem. This was achieved through critical computational optimizations, including the use of NumPy vectorized matrix operations to process 500 hypothesis particles simultaneously and the implementation of an $O(1)$ distance transform lookup for ultrasonic likelihood scoring. Furthermore, the interception-based experimental topology ensured high scientific validity, allowing for perfectly synchronized, parameter-driven comparative analysis between the proposed model and existing baselines.

Ultimately, this work reinforces a fundamental principle in robotic systems and networks: the intelligence of the fusion logic and real-time confidence scaling are more critical to system survival than raw hardware data density. The ability of the DA-PF to maintain localization using inexpensive hardware through “trust management” provides a scalable framework for deploying resilient autonomous systems in unpredictable, real-world environments.

REFERENCES

- [1] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005.
- [2] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and K. Woodall, “Robot Operating System 2: Design, architecture, and rigorous testing,” *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.
- [3] S. Macenski, F. Martin, R. White, and J. Clavero, “The Marathon 2: A navigation system,” in *Proc. IEEE/RSJ IROS*, Las Vegas, NV, 2020, pp. 2494–2501.
- [4] D. Fox, “Adapting the sample size in particle filters through KLD-sampling,” *Int. J. Robotics Research*, vol. 22, no. 12, pp. 985–1003, 2003.
- [5] M. Grupp, “evo: Python package for the evaluation of odometry and SLAM,” <https://github.com/MichaelGrupp/evo>, 2017.
- [6] P. Virtanen et al., “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.
- [7] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez, “Automatic generation and detection of highly reliable fiducial markers under occlusion,” *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014.
- [8] F. Dellaert, D. Fox, W. Burgard, and S. Thrun, “Monte Carlo localization for mobile robots,” in *Proc. IEEE ICRA*, Detroit, MI, 1999, pp. 1322–1328.